

Microsoft Azure

Search resources, services, and docs (G+)

Copilot

tredence23@michaelklo...
DEFAULT DIRECTORY (MICHAELKLO...)

Home > vaishnavidata
Storage account

Audit disaster recovery readiness for this storage account Improve storage account data protection How many critical alerts for this storage account

Search

Upload Open in Explorer Delete Move Refresh Open in mobile CLI / PS Feedback

Overview

- Activity log
- Tags
- Diagnose and solve problems
- Access Control (IAM)
- Data migration
- Events
- Storage browser
- Partner solutions
- Resource visualizer
- Data storage
 - Containers
 - File shares
 - Queues
 - Tables

Essentials

Resource group (move)
LabsKraft2027

Location
southindia

Subscription (move)
LabsKraft

Subscription ID
d008c7cc-9795-4292-bf79-74ae52cda63d

Disk state
Available

Tags (edit)
Add tags

Performance
Standard

Replication
Locally-redundant storage (LRS)

Account kind
StorageV2 (general purpose v2)

Provisioning state
Succeeded

Created
8/25/2025, 10:08:10 AM

JSON View

Properties Monitoring Capabilities (5) Recommendations (0) Tutorials Tools + SDKs

Data Lake Storage

Hierarchical namespace Enabled

Require secure transfer for REST API Enabled

Security

https://portal.azure.com/?nopreview=true#@michaelklo...@michaelklo...onmicrosoft.com/resource/subscriptions/d008c7cc-9795-4292-bf79-74ae52cda63d/resourcegroups/LabsKraft2027/providers/Microsoft.Storage/storageAccount...

Step 1: Create a Storage Account -> container inside it with the name capstoneproject.

Microsoft Azure

Search resources, services, and docs (G+)

Copilot

tredence23@michaelklo...
DEFAULT DIRECTORY (MICHAELKLO...)

Home > vaishnavidata
Storage account

Containers

Search containers by prefix

Only show active containers

Showing all 2 items

Name	Last modified	Anonymous access level	Lease state
\$logs	8/25/2025, 10:08:45 AM	Private	Available
capstoneproject	8/25/2025, 10:12:56 AM	Private	Available

Access Control (IAM)

Data migration

Events

Storage browser

Partner solutions

Resource visualizer

Data storage

Containers

File shares

Queues

Tables

Security + networking

Networking

Access keys

Shared access signature

Add or remove favorites by pressing Ctrl+Shift+F

Step 2: Create unity catalog and for that make access connector and give required permissions using IAM in storage account.

Microsoft Azure

Search resources, services, and docs (G+)

Copilot

9

tredecence23@michaelklo...
DEFAULT DIRECTORY (MICHAELK...

Home > AccessConnectorCreate-20250825101652 | Overview > LabsKraft2027 >

vaishnavi_connect

Access Connector for Azure Databricks

✦ ☆ ...

×

Search

Refresh

Delete

Overview

Activity log

Access control (IAM)

Tags

Resource visualizer

Settings

Automation

Help

Essentials

Resource group (move)
LabsKraft2027

Location
South India

Subscription (move)
LabsKraft

Subscription ID
d008c7cc-9795-4292-bf79-74ae52cda63d

Tags (edit)
Add tags

State
Succeeded

Resource ID
/subscriptions/d008c7cc-9795-4292-bf79-74ae52cda63d/resourceGroups/...

JSON View

Add or remove favorites by pressing Ctrl+Shift+F

Storage account(vaishnavidata) -> IAM -> add role -> Storage Blob Data Contributor -> member->

Microsoft Azure

Search resources, services, and docs (G+)

Copilot

9

tredecence23@michaelklo...
DEFAULT DIRECTORY (MICHAELK...

Home > vaishnavidata | Access Control (IAM) >

Add role assignment

...

×

Role

Members

Conditions

Review + assign

A role definition is a collection of permissions. You can use the built-in roles or you can create your own custom roles. [Learn more](#)

Job function roles

Privileged administrator roles

Grant access to Azure resources based on job function, such as the ability to create virtual machines.

storage blob

Type : All

Category : All

Name ↑↓	Description ↑↓	Type ↑↓	Category ↑↓	Details
CreateBlobStorage		CustomRole	None	View
Defender CSPM Storage Data Scan...	Grants access to read blobs and files. This role is used by the data scanner of Dfender CSPM.	BuiltInRole	None	View
Defender for Storage Data Scanner	Grants access to read blobs and update index tags. This role is used by the data scanner of...	BuiltInRole	None	View
Storage Blob Data Contributor	Allows for read, write and delete access to Azure Storage blob containers and data	BuiltInRole	Storage	View
Storage Blob Data Owner	Allows for full access to Azure Storage blob containers and data, including assigning POSI...	BuiltInRole	Storage	View
Storage Blob Data Reader	Allows for read access to Azure Storage blob containers and data	BuiltInRole	Storage	View
Storage Blob Delegator	Allows for generation of a user delegation key which can be used to sign SAS tokens	BuiltInRole	Storage	View

Review + assign

Previous

Next

Feedback

Microsoft Azure

Search resources, services, and docs (G+/)

Copilot

tredence23@michaelklo...
DEFAULT DIRECTORY (MICHAELK...

Home > vaishnavidata | Access Control (IAM) >

Add role assignment

RoleMembersConditionsReview + assign

Showing a filtered list of roles because your permissions include a condition. [Learn more](#)
[View my access](#)

Selected role

Storage Blob Data Contributor

Assign access to

User, group, or service principal

Managed identity

Members

+ Select members

Description

Optional

Review + assign

Previous

Next

Select managed identities

Some results might be hidden due to your ABAC condition.

Subscription *

LabsKraft

Managed identity

Access Connector for Azure Databricks (8)

Search by name

sanyam-connector
/subscriptions/d008c7cc-9795-4292-bf79-74ae52cda63d/resourceGroups/Labs...

shibang
/subscriptions/d008c7cc-9795-4292-bf79-74ae52cda63d/resourceGroups/Labs...

shivam-connector
/subscriptions/d008c7cc-9795-4292-bf79-74ae52cda63d/resourceGroups/Labs...

Selected members:

vaishnavi_connect
/subscriptions/d008c7cc-9795-4292-bf79-74ae52cda63d/resourceGroups/...

Remove

Select

Close

Feedback

Microsoft Azure

databricks

Search data, notebooks, recents, and more...

CTRL + P

tred_d6

New

Workspace

Recents

Catalog

Jobs & Pipelines *

Compute

Marketplace

SQL

SQL Editor

Queries

Dashboards

Genie

Alerts

Query History

SQL Warehouses

Data Engineering

Job Runs

Data Ingestion

AI/ML

External Locations >

Create a new external location

An external location allows you to access your data stored in cloud storage (e.g. Azure Data Lake Storage). You will need the cloud storage path and a paired credential (e.g. managed identity) which gives access to that path [Learn more](#)

External location name*

vm_retail

Storage type*

Azure Data Lake Storage

URL* ⓘ

Enter the bucket path that you want to use as the external location. Note: This must be an ADLS Gen2 storage account with a hierarchical namespace

abfss://capstoneproject@vaishnavidata.dfs.core.windows.net/

Copy from DBFS

Storage credential* [Learn more](#)

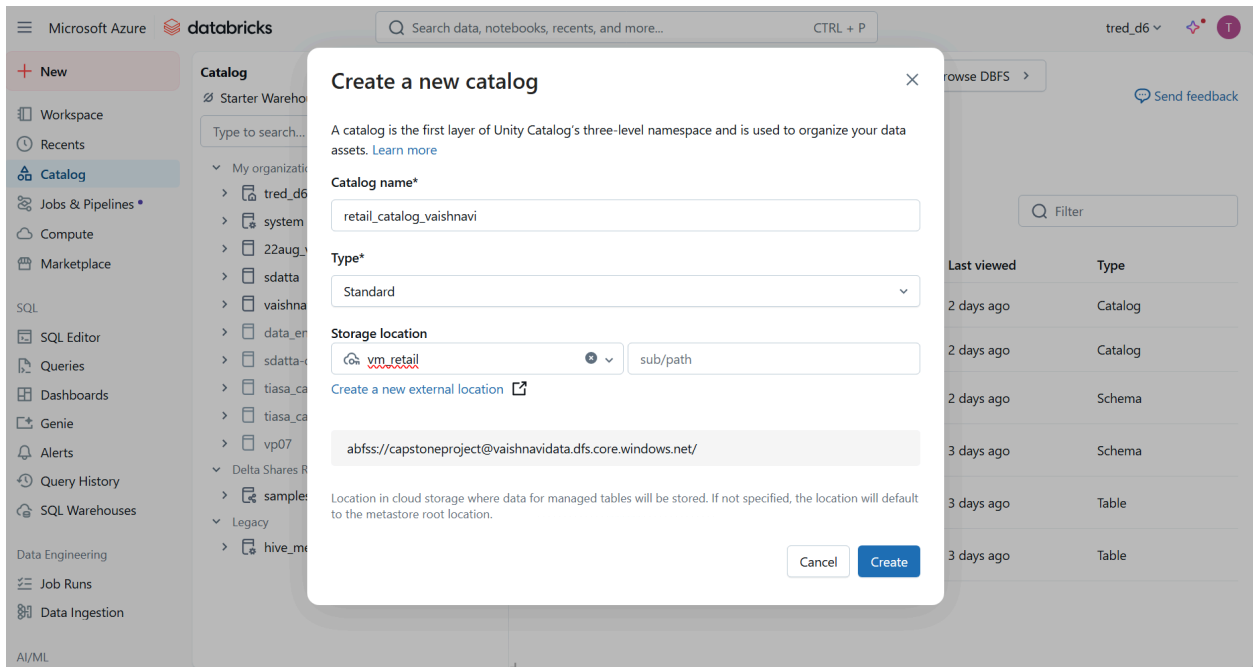
Provide a storage credential capable of accessing the URL

+ Create new storage credential

Access connector ID [Learn more](#)

/subscriptions/d008c7cc-9795-4292-bf79-74ae52cda63d/resourceGroups/LabsKraft2027/providers/Microsoft.Databricks/acc

User assigned managed identity ID (optional)



```
%sql
USE CATALOG retail_catalog_vaishnavi;

CREATE SCHEMA IF NOT EXISTS bronze;
CREATE SCHEMA IF NOT EXISTS silver;
CREATE SCHEMA IF NOT EXISTS gold;
```

_yt8Q~QsF1J~SxYRNQYTfugj.wsggsB-9BS9zaZ9

The screenshot displays the Databricks Jobs & Pipelines interface. The top navigation bar includes 'Microsoft Azure', 'databricks', a search bar, and a user profile 'tred_d6'. The left sidebar lists various workspace components like 'Workspace', 'Recents', 'Catalog', 'Jobs & Pipelines' (selected), 'Compute', 'Marketplace', 'SQL', 'SQL Editor', 'Queries', 'Dashboards', 'Genie', 'Alerts', 'Query History', 'SQL Warehouses', 'Data Engineering', 'Job Runs', 'Data Ingestion', 'AI/ML', and 'Playground'. The main area shows a pipeline named 'vaishnavi-aug-25' in the 'Development' environment, which has been 'Completed' on '8/25/2025, 3:05:41 PM'. The pipeline graph consists of several jobs: 'bronze_orders', 'silver_orders', 'gold_anomaly_tre...', 'gold_conversion_ju...', 'gold_sales_region', 'gold_store_tiers', and 'gold_weight'. The right sidebar provides 'Pipeline details' including the 'Pipeline ID' (143e1c99-e538-4b10-bc07-b34351860dea), 'Pipeline type' (ETL pipeline), 'Source code' (a GitHub link), 'Run as' (Tredence LabsKraft23), and 'Tags' (None). A 'View logs' button is located at the bottom right of the pipeline details section.

1. Storage & Environment Setup

- Configured **Azure Data Lake Gen2 (ADLS Gen2)** access with Spark.
- Storage details:
 - Storage Account: **vaishnavidata**
 - Container: **capstoneproject**

Configured **access key authentication** via Spark:

```
spark.conf.set(
  "fs.azure.account.key.vaishnavidata.dfs.core.windows.net",
  "<storage_account_key>"
)
```

◦

2. Bronze Layer (Raw Ingestion)

- Defined **schema** for incoming retail orders (POS + IoT signals).
- Used **DLT + Auto Loader (cloudFiles)** to ingest raw CSVs from path:

```
abfss://capstoneproject@vaishnavidata.dfs.core.windows.net/incoming/
```

- Saved into Delta table:
`retail_catalog_vaishnavi.bronze.bronze_orders`
- Key features:
 - Streaming ingestion (`readStream`)
 - Schema tracking (`schema_path` in ADLS)
 - Fields: OrderID, CustomerID, Region, StoreID, Product, Quantity, UnitPrice, SalesAmount, OrderTime, Device details, Anomaly flags

3. Silver Layer (Cleansed & Standardized)

- Created `retail_catalog_vaishnavi.silver.silver_orders` with transformations:
 - Casted numeric fields to correct types (Quantity, Price, SalesAmount, Temperature).
 - Converted `OrderTime` → timestamp.
 - Deduplicated records on **OrderID**.
 - Handled missing `AnomalyType` (set as "Unknown" if anomaly present but type missing).

- Standardized fields (uppercase StoreID, Product, DeviceStatus).
 - Outcome: **cleaned, deduped dataset ready for analytics.**
-

4. Gold Layer (Analytics & Business KPIs)

Created multiple curated analytical tables for BI/dashboarding:

1. **gold_sales_region**

- Daily + Monthly sales trends by Region.
- Aggregated **TotalSales**.

2. **gold_anomaly_trends**

- Device anomaly counts grouped by Store, Date, DeviceType, and AnomalyType.
- Used for operational monitoring.

3. **gold_conversion_impact**

- Analyzed failure rate due to device anomalies.
- Computed **FailureRate = FailedOrders / TotalOrders**.

4. **gold_store_tiers**

- Classified stores into **High, Medium, Low performers** based on **TotalSales**.

5. **gold_weighted_sales_anomalies**

- Calculated **Sales per Anomaly** (impact measure of IoT failures).

All tables materialized under:

retail_catalog_vaishnavi.gold.*

```
import pyspark.sql.functions as F
from pyspark.sql.window import Window
```

```

import dlt
from pyspark.sql.types import *

schema_path
="abfss://capstoneproject@vaishnavidata.dfs.core.windows.net/dlt_schemas/bronze
_orders_schema"
schema = StructType([
    StructField("OrderID", StringType()),
    StructField("CustomerID", StringType()),
    StructField("Region", StringType()),
    StructField("StoreID", StringType()),
    StructField("Product", StringType()),
    StructField("Quantity", IntegerType()),
    StructField("UnitPrice", DoubleType()),
    StructField("SalesAmount", DoubleType()),
    StructField("OrderTime", TimestampType()),
    StructField("DeviceType", StringType()),
    StructField("Temperature", DoubleType()),
    StructField("DeviceStatus", StringType()),
    StructField("AnomalyFlag", StringType(), True),
    StructField("AnomalyType", StringType(), True)
    # Add all columns as per your CSV structure
])
@dlt.table(
    name="retail_catalog_vaishnavi.bronze.bronze_orders",
    comment="Raw retail + IoT data ingested from ADLS Gen2"
)
def bronze_orders():
    return (
        spark.readStream.format("cloudFiles")
        .option("cloudFiles.format", "csv")
        .option("header", "true")
        .option("cloudFiles.schemaLocation", schema_path)
        .schema(schema)

    .load("abfss://capstoneproject@vaishnavidata.dfs.core.windows.net/incoming/")
    )

```



```

@dlt.table(
    name="retail_catalog_vaishnavi.silver.silver_orders",
    comment="Silver layer: cleaned and deduplicated IoT orders with anomaly
handling"
)
def silver_orders():
    # Read Bronze
    orders = dlt.read("retail_catalog_vaishnavi.bronze.bronze_orders")

    # Standardize schema
    orders = (
        orders.withColumn("Quantity", F.col("Quantity").cast(IntegerType()))
            .withColumn("UnitPrice", F.col("UnitPrice").cast(DoubleType()))
            .withColumn("SalesAmount",
F.col("SalesAmount").cast(DoubleType()))
            .withColumn("Temperature",
F.col("Temperature").cast(DoubleType()))
            .withColumn("OrderTime", F.to_timestamp("OrderTime", "M/d/yyyy
H:mm"))
    )

    # Deduplicate by OrderID
    orders = orders.dropDuplicates(["OrderID"])

    # Handle missing anomaly type
    orders = orders.withColumn(
        "AnomalyType",
        F.when((F.col("AnomalyFlag") == 1) & (F.col("AnomalyType").isNull()),
"Unknown")
            .when((F.col("AnomalyFlag") == 1) & (F.col("AnomalyType") == ""),
"Unknown")
            .otherwise(F.col("AnomalyType"))
    )

    # Add business rule fields
    orders = (
        orders.withColumn("IsAnomaly", F.col("AnomalyFlag") == 1)
            .withColumn("DeviceStatus", F.upper(F.col("DeviceStatus")))
    )

```

```

        .withColumn("NormalizedStoreID", F.upper(F.col("StoreID")))
        .withColumn("NormalizedProduct", F.upper(F.col("Product")))
    )

    return orders

```

```

from dlt import table, read
import pyspark.sql.functions as F

# =====
# Gold Layer Tables (capstone_retail.gold)
# =====

# 1. Daily / Monthly Sales per Region
@table(
    name="retail_catalog_vaishnavi.gold.gold_sales_region",
    comment="Gold table - Aggregated store sales and anomalies"
)

def gold_sales_region():
    return (
        read("retail_catalog_vaishnavi.silver.silver_orders")
        .withColumn("OrderDate", F.to_date("OrderTime", "M/d/yyyy H:mm"))
        .withColumn("YearMonth", F.date_format("OrderDate", "yyyy-MM"))
        .groupBy("Region", "OrderDate", "YearMonth")
        .agg(F.sum("SalesAmount").alias("TotalSales"))
    )

# 2. Device Anomaly Trend per Store
@table(
    name="retail_catalog_vaishnavi.gold.gold_anomaly_trends",
    comment="Device Anomaly Trend per Store"
)

def gold_anomaly_trends():
    return (
        read("retail_catalog_vaishnavi.silver.silver_orders")
        .withColumn("OrderDate", F.to_date("OrderTime", "M/d/yyyy H:mm"))
        .groupBy("StoreID", "OrderDate", "DeviceType", "AnomalyType")

```

```

        .agg(F.count("*").alias("AnomalyCount"))
    )

# 3. Conversion Rate Impact when devices fail
@table(
    name="retail_catalog_vaishnavi.gold.gold_conversion_impact",
    comment="Conversion rate impact of device failures"
)

def gold_conversion_impact():
    df = read("retail_catalog_vaishnavi.silver.silver_orders")
    total_orders = df.groupBy("StoreID").agg(F.count("*").alias("TotalOrders"))
    failed_orders = (
        df.filter(df.AnomalyFlag == 1)
        .groupBy("StoreID")
        .agg(F.count("*").alias("FailedOrders"))
    )

    return (
        total_orders.join(failed_orders, "StoreID", "left")
        .withColumn("FailedOrders", F.coalesce(F.col("FailedOrders"),
F.lit(0)))
        .withColumn("FailureRate", F.col("FailedOrders") /
F.col("TotalOrders"))
    )

# 4. Store Tier Classification
@table(
    name="retail_catalog_vaishnavi.gold.gold_store_tiers",
    comment="Store tiers based on total sales (High / Medium / Low performers)"
)

def gold_store_tiers():
    df = (
        read("retail_catalog_vaishnavi.silver.silver_orders")
        .groupBy("StoreID")
        .agg(F.sum("SalesAmount").alias("TotalSales"))
    )

    return (
        df.withColumn(

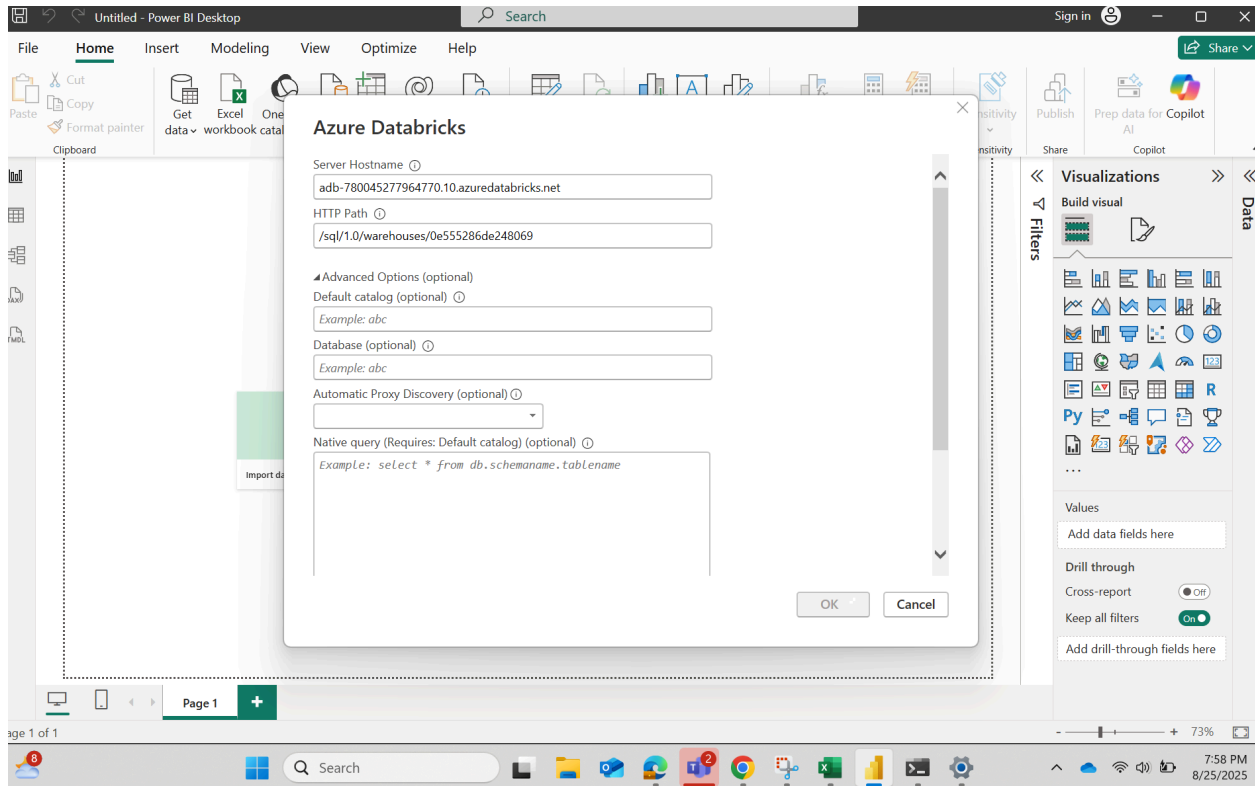
```

```

        "StoreTier",
        F.when(F.col("TotalSales") > 100000, "High")
          .when(F.col("TotalSales") > 50000, "Medium")
          .otherwise("Low")
      )
    )

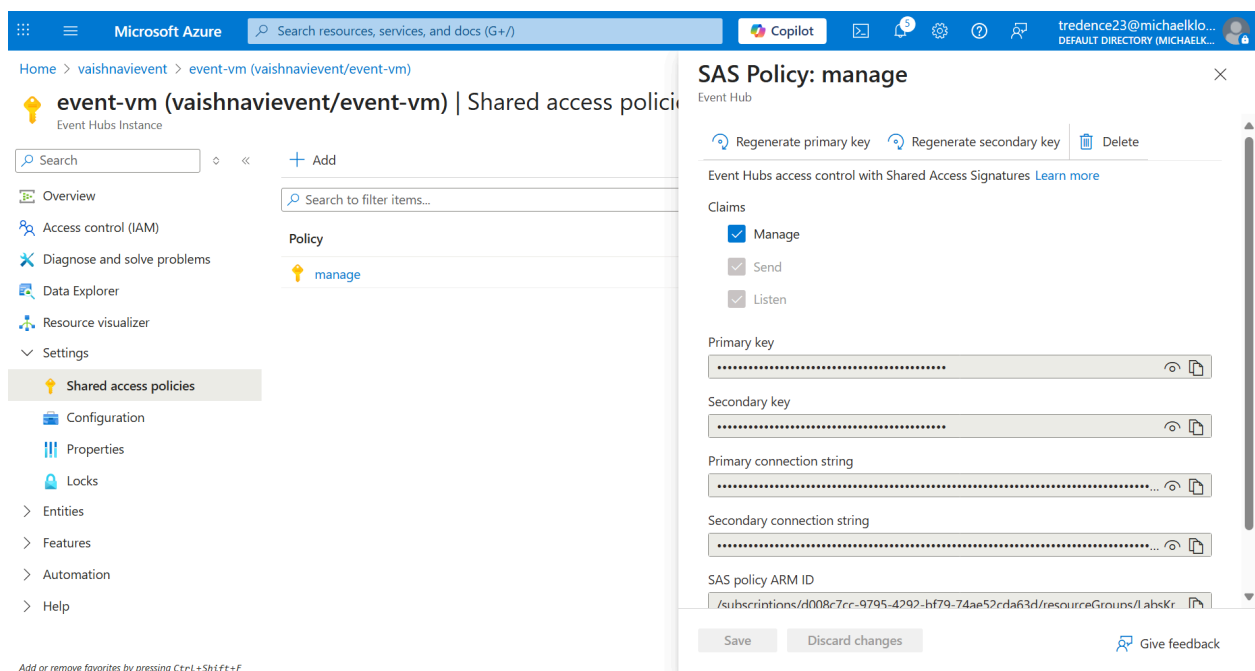
# 5. Weighted Average Sales vs Anomalies
@table(
  name="retail_catalog_vaishnavi.gold.gold_weighted_sales_anomalies",
  comment="Weighted average of sales vs anomalies per store"
)
def gold_weighted_sales_anomalies():
  df = read("retail_catalog_vaishnavi.silver.silver_orders")
  return (
    df.groupBy("StoreID")
      .agg(
        F.sum("SalesAmount").alias("TotalSales"),
        F.sum(F.when(F.col("AnomalyFlag") == 1,
1).otherwise(0)).alias("TotalAnomalies")
      )
      .withColumn("SalesPerAnomaly",
        F.when(F.col("TotalAnomalies") > 0, F.col("TotalSales") /
F.col("TotalAnomalies"))
        .otherwise(F.lit(None))
      )
    )
  )

```



PowerBI Not working

Event-Hub



->Create VM-Machine

In the command prompt-

```
sudo apt update
```

```
sudo apt install python3 python3-pip -y
```

```
sudo apt update
```

```
sudo apt install -y python3-pandas
```

```
nano iot_stream.py
```

```
import random
```

```
import time
```

```
import uuid
```

```
from datetime import datetime
```

```
from azure.eventhub import EventHubProducerClient, EventData
```

```
# -----
```

```
# CONFIGURATION
```

```
# -----
```

```
EVENT_HUB_CONNECTION_STR =
```

```
"Endpoint=sb://hetulproject01eventspace.servicebus.windows.net/;SharedAccessKeyName=hello;S  
haredAccessKey=Cqw6MOblR0mllhVGqOjbbEtH6s6lvgl2+AEhDNW7r4=;EntityPath=hetulhub"
```

```
EVENT_HUB_NAME = "hetulhub"
```

```
# -----
```

```
# Function to generate one IoT row
```

```
# -----
```

```
def generate_row():
```

```
    quantity = random.randint(1, 10)
```

```
    unit_price = round(random.uniform(50, 500), 2)
```

```
    sales_amount = round(quantity * unit_price, 2)
```

```
    device_status = random.choice(["Active", "Inactive", "Error"])
```

```
    anomaly_flag = 1 if device_status != "Active" else 0
```

```
    anomaly_type = random.choice(["Overheat", "LowBattery", "Disconnected"]) if anomaly_flag  
else "None"
```

```
    return {
```

```
        "OrderID": str(uuid.uuid4()),
```

```
        "CustomerID": f"CUST{random.randint(1000,9999)}",
```

```
        "Region": random.choice(["North", "South", "East", "West"]),
```

```

"StoreID": random.choice(["Store001", "Store002", "Store003"]),
"Product": random.choice(["Laptop", "Phone", "Monitor", "Keyboard", "Mouse"]),
"Quantity": quantity,
"UnitPrice": unit_price,
"SalesAmount": sales_amount,
"OrderTime": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
"DeviceType": random.choice(["SensorA", "SensorB", "SensorC"]),
"Temperature": round(random.uniform(20, 100), 2),
"DeviceStatus": device_status,
"AnomalyFlag": anomaly_flag,
"AnomalyType": anomaly_type
}

```

```

# -----
# Event Hub Producer
# -----
producer = EventHubProducerClient.from_connection_string(
    conn_str=EVENT_HUB_CONNECTION_STR,
    eventhub_name=EVENT_HUB_NAME
)

# -----
# Send data every second
# -----
try:
    for _ in range(1000): # Send exactly 1000 rows
        row = generate_row()
        event_data = EventData(str(row)) # convert dict to string
        with producer:
            producer.send_batch([event_data])
            print(f"Sent row: {row}")
            time.sleep(1) # 1-second interval
except KeyboardInterrupt:
    print("Streaming stopped by user.")
finally:
    producer.close()

```

pip install pandas azure-eventhub

Microsoft Azure Search resources, services, and docs (G+/) Copilot tredence23@michaelklo... DEFAULT DIRECTORY (MICHAELKLO...)

Home > Event Hubs > vaishnavievent

vaishnavievent | Data Explorer

Event Hubs Namespace

Search

- Overview
- Activity log
- Access control (IAM)
- Tags
- Diagnose and solve problems
- Data Explorer**
- Resource visualizer
- Events
- Settings
- Entities
- Monitoring
- Automation
- Help

Event Hub *
event-vm
[Create a new Event Hub](#)

Transmit prepared or custom data
[Send events](#)

Inspect your data with following properties

Partition ID *
All partition IDs

Consumer group *
\$Default

Event position
☒ Oldest position ⓘ
☐ Custom position ⓘ
☐ Newest position ⓘ

> Advanced properties

Clear all

Auth settings Learn more Give feedback

Total received events : 17 [View next events](#) [Export events](#)

Sequence Nu...	Offset	Partition ID	Enqueued Time	Content Type
785	336984	0	Mon, Aug 25, 25, 09:02:52 PM G...	
786	337416	0	Mon, Aug 25, 25, 09:02:54 PM G...	
787	337848	0	Mon, Aug 25, 25, 09:02:55 PM G...	
788	338280	0	Mon, Aug 25, 25, 09:02:57 PM G...	
789	338712	0	Mon, Aug 25, 25, 09:02:58 PM G...	
790	339144	0	Mon, Aug 25, 25, 09:03:00 PM G...	
791	339576	0	Mon, Aug 25, 25, 09:03:02 PM G...	
792	340008	0	Mon, Aug 25, 25, 09:03:03 PM G...	
793	340432	0	Mon, Aug 25, 25, 09:03:05 PM G...	
794	340856	0	Mon, Aug 25, 25, 09:03:06 PM G...	

Event body Event properties

Add or remove favorites by pressing Ctrl+Shift+F

```
# =====
# Bronze → Silver → Gold with Event Hub Integration
# =====
```

```
from dlt import table, read
import pyspark.sql.functions as F
from pyspark.sql.types import *
```

```
# =====
# ① Bronze Layer: Schema & CSV
# =====

iot_schema = StructType([
    StructField("OrderID", StringType(), True),
    StructField("CustomerID", StringType(), True),
    StructField("Region", StringType(), True),
    StructField("StoreID", StringType(), True),
    StructField("Product", StringType(), True),
    StructField("Quantity", IntegerType(), True),
    StructField("UnitPrice", DoubleType(), True),
    StructField("SalesAmount", DoubleType(), True),
    StructField("OrderTime", StringType(), True),
])
```



```

    StructField("DeviceType", StringType(), True),
    StructField("Temperature", DoubleType(), True),
    StructField("DeviceStatus", StringType(), True),
    StructField("AnomalyFlag", IntegerType(), True),
    StructField("AnomalyType", StringType(), True)
])

raw_path =
"abfss://capstoneproject@vaishnavidata.dfs.core.windows.net/incoming/"
schema_path =
"abfss://capstoneproject@vaishnavidata.dfs.core.windows.net/dlt_schemas/bronze_
orders_vaishnavi"

@table(
    name="retail_catalog_vaishnavi.bronze.bronze_orders_csv",
    comment="Bronze table - raw IoT CSV"
)

def bronze_orders_csv():
    return (
        spark.readStream
            .format("cloudFiles")
            .option("cloudFiles.format", "csv")
            .option("header", True)
            .option("cloudFiles.schemaLocation", schema_path)
            .schema(iot_schema)
            .load(raw_path)
            .withColumn("OrderTime", F.to_timestamp("OrderTime"))
    )

# =====
# ② Bronze Layer: Event Hub IoT
# =====

eventhub_connection_string =
"Endpoint=sb://vaishnavievent.servicebus.windows.net/;SharedAccessKeyName=manag
e;SharedAccessKey=FKKqGhzAyDEqYXgNZvxzpYat2gUjWlVTS+AEhAIkesY;;EntityPath=event
-vm"

consumer_group = "$Default"

```

```

ehConf = {
    'eventhubs.connectionString': eventhub_connection_string,
    'eventhubs.consumerGroup': consumer_group,
    'eventhubs.startingPosition': '@latest'
}

@table(
    name="retail_catalog_vaishnavi.bronze.bronze_orders_eventhub",
    comment="Bronze table - live IoT data from Event Hub"
)

def bronze_orders_eventhub():
    df = spark.readStream \
        .format("eventhubs") \
        .options(**ehConf) \
        .load()

    # Event Hub body is bytes → convert to string → parse JSON
    df1 = df.selectExpr("cast(body as string) as json_string")
    df_parsed = df1.select(F.from_json(F.col("json_string"),
iot_schema).alias("data")).select("data.*")
    return df_parsed.withColumn("OrderTime", F.to_timestamp("OrderTime"))

# =====
# ③ Unified Bronze Table
# =====

@table(
    name="retail_catalog_vaishnavi.bronze.bronze_orders_1",
    comment="Unified Bronze table (CSV + Event Hub)"
)

def bronze_orders():
    df_csv = read("retail_catalog_vaishnavi.bronze.bronze_orders_csv")
    df_eh = read("retail_catalog_vaishnavi.bronze.bronze_orders_eventhub")
    return df_csv.unionByName(df_eh)

# =====
# ④ Silver Layer
# =====

@table(
    name="retail_catalog_vaishnavi.silver.silver_orders_1",

```

```

    comment="Silver layer: cleaned and deduplicated IoT orders with anomaly
handling"
)
def silver_orders():
    orders = read("retail_catalog_vaishnavi.bronze.bronze_orders_1")

    orders = (
        orders.withColumn("Quantity", F.col("Quantity").cast(IntegerType()))
            .withColumn("UnitPrice", F.col("UnitPrice").cast(DoubleType()))
            .withColumn("SalesAmount",
F.col("SalesAmount").cast(DoubleType()))
            .withColumn("Temperature",
F.col("Temperature").cast(DoubleType()))
            .withColumn("OrderTime", F.to_timestamp("OrderTime", "M/d/yyyy
H:mm"))
    )

    orders = orders.dropDuplicates(["OrderID"])

    orders = orders.withColumn(
        "AnomalyType",
        F.when((F.col("AnomalyFlag") == 1) & (F.col("AnomalyType").isNull()),
"Unknown")
            .when((F.col("AnomalyFlag") == 1) & (F.col("AnomalyType") == ""),
"Unknown")
            .otherwise(F.col("AnomalyType"))
    )

    orders = (
        orders.withColumn("IsAnomaly", F.col("AnomalyFlag") == 1)
            .withColumn("DeviceStatus", F.upper(F.col("DeviceStatus")))
            .withColumn("NormalizedStoreID", F.upper(F.col("StoreID")))
            .withColumn("NormalizedProduct", F.upper(F.col("Product")))
    )

    return orders

# =====

```

```

# 5 Gold Layer
# =====

@table(
    name="retail_catalog_vaishnavi.gold.gold_orders_1",
    comment="Daily & Monthly Sales per Region"
)

def gold_sales_region():
    return (
        read("retail_catalog_vaishnavi.silver.silver_orders_1")
        .withColumn("OrderDate", F.to_date("OrderTime", "M/d/yyyy H:mm"))
        .withColumn("YearMonth", F.date_format("OrderDate", "yyyy-MM"))
        .groupBy("Region", "OrderDate", "YearMonth")
        .agg(F.sum("SalesAmount").alias("TotalSales"))
    )

@table(
    name="retail_catalog_vaishnavi.gold.gold_anomaly_trends_1",
    comment="Device Anomaly Trend per Store"
)

def gold_anomaly_trends():
    return (
        read("retail_catalog_vaishnavi.silver.silver_orders_1")
        .withColumn("OrderDate", F.to_date("OrderTime", "M/d/yyyy H:mm"))
        .groupBy("StoreID", "OrderDate", "DeviceType", "AnomalyType")
        .agg(F.count("*").alias("AnomalyCount"))
    )

@table(
    name="retail_catalog_vaishnavi.gold.gold_conversion_impact_1",
    comment="Conversion rate impact of device failures"
)

def gold_conversion_impact():
    df = read("retail_catalog_vaishnavi.silver.silver_orders_1")
    total_orders = df.groupBy("StoreID").agg(F.count("*").alias("TotalOrders"))
    failed_orders = (
        df.filter(df.AnomalyFlag == 1)
        .groupBy("StoreID")
        .agg(F.count("*").alias("FailedOrders"))
    )

```

```

    )
    return (
        total_orders.join(failed_orders, "StoreID", "left")
        .withColumn("FailedOrders", F.coalesce(F.col("FailedOrders"),
F.lit(0)))
        .withColumn("FailureRate", F.col("FailedOrders") /
F.col("TotalOrders"))
    )

@table(
    name="retail_catalog_vaishnavi.gold.gold_store_tiers_1",
    comment="Store tiers based on total sales (High / Medium / Low performers)"
)
def gold_store_tiers():
    df = (
        read("retail_catalog_vaishnavi.silver.silver_orders_1")
        .groupBy("StoreID")
        .agg(F.sum("SalesAmount").alias("TotalSales"))
    )
    return (
        df.withColumn(
            "StoreTier",
            F.when(F.col("TotalSales") > 100000, "High")
            .when(F.col("TotalSales") > 50000, "Medium")
            .otherwise("Low")
        )
    )

@table(
    name="retail_catalog_vaishnavi.gold.gold_weighted_sales_anomalies_1",
    comment="Weighted average of sales vs anomalies per store"
)
def gold_weighted_sales_anomalies():
    df = read("retail_catalog_vaishnavi.silver.silver_orders_1")
    return (
        df.groupBy("StoreID")
        .agg(
            F.sum("SalesAmount").alias("TotalSales"),

```

```
        F.sum(F.when(F.col("AnomalyFlag") == 1,
1).otherwise(0)).alias("TotalAnomalies")
    )
    .withColumn("SalesPerAnomaly",
        F.when(F.col("TotalAnomalies") > 0, F.col("TotalSales") /
F.col("TotalAnomalies"))
        .otherwise(F.lit(None))
    )
)
```